

Lab Assignment 3

High-Dimensional Vector Range Search and Algorithm Analysis

Student ID: 411321344

Name: Wilkend AUGUSTE

Course: Lab of Algorithm

Date: March 20, 2026



National Dong Hwa University

1. Problem Understanding and Objectives

For this lab assignment, we need to implement a range search on high-dimensional vectors. Given a database of vectors, a query point q , and a radius r , we want to find all vectors within distance r from q .

In math terms, we want all v_i where:

$$\text{dist}(q, v_i) \leq r$$

I used Euclidean distance: $\text{dist}(a, b) = \sqrt{[\sum (a_i - b_i)^2]}$. This kind of search is used in AI systems like vector databases, recommendation systems, and RAG pipelines. The main things I wanted to observe were:

- How search time grows as we increase the number of vectors (n)
- What happens to performance as dimensions (d) increase
- How the search radius (r) affects query time and results found
- Whether KD-tree is actually faster than brute-force in practice

2. Data Generation

I generated all the vector data in my C++ program using `std::mt19937` with a fixed seed of 42 for reproducibility. All values are drawn from a uniform distribution between 0.0 and 10.0.

Parameter	Setting
Random generator	<code>std::mt19937</code> (Mersenne Twister)
Distribution	Uniform [0.0, 10.0]
Data seed	42 (+ repetition)
Query seed	200 (+ repetition)
Data sizes (n)	1,000 / 5,000 / 10,000 / 50,000 / 100,000
Dimensions (d)	4, 8, 16, 32, 64, 128
Queries per setting	25
Repetitions	3

3. Algorithm Design

3.1 Brute-Force Search

The brute-force method goes through every vector in the database and checks the distance to the query. Simple to implement and always correct but checks all n vectors every time.

Pseudocode:

```
results = []
for each vector v in database:
    if dist (query, v) <= r:
        results.add(v)
return results
```

Time complexity: $O(n \times d)$.

3.2 KD-Tree (Divide and Conquer)

I implemented a KD-tree that splits the data space recursively by alternating dimensions. At each level, we split by median value. During search, the key optimization is pruning: if the query's distance to the splitting hyperplane is larger than r , the entire subtree on the far side is skipped.

Build pseudocode:

```
function build(points, depth):
    dim = depth mod d
    sort points by dim
    pick median as node
    node.left = build(left half)
    node.right = build(right half)
```

Search pseudocode:

```
function search (node, query, r):
    if dist (node, query) <= r: add to results
    search closer side always
    if |diff| <= r: search far side too    // pruning
```

Build: $O(n \log n)$. Query: $O(\log n)$ average for low-d, degrades to $O(n)$ in high-d.

4. Time Complexity Analysis

Method	Phase	Complexity	Explanation
Brute-Force	Query	$O(n \times d)$	Checks every vector, d ops each
KD-Tree	Build	$O(n \log n)$	Sorting at each recursive level
KD-Tree	Query (low d)	$O(\log n)$	Pruning works well
KD-Tree	Query (high d)	$O(n)$	Pruning stops working

In theory KD-tree should be much faster for queries, but this only holds for low dimensions. When d gets large, the pruning condition almost never activates.

5. Experimental Design

Item	Details
Language	C++20
Compile	<code>g++ -std=c++20 lab3_search.cpp -o lab3_search</code>
Timer	<code>std::chrono::high_resolution_clock</code>
Queries / setting	25
Repetitions	3 (averaged)
Exp 1	Vary $n \in \{1000, 5000, 10000, 50000, 100000\}$, $d=16$, $r=1.5$
Exp 2	Vary $d \in \{4, 8, 16, 32, 64, 128\}$, $n=10000$, $r=1.5$
Exp 3	Vary $r \in \{0.5, 1.5, 3.0\}$, $n=10000$, $d=16$

6. Experimental Results and Analysis

6.1 Experiment 1 — Varying Dataset Size ($d=16$, $r=1.5$)

Experiment 1: Varying Dataset Size ($d=16$, $r=1.5$)					
Param	BF Query (ms)	KD Build (ms)	KD Query (ms)	BF Found	KD Found
1000	0.0404	3.68	0.0041	0	0
5000	0.6139	57.05	0.0153	0	0
10000	1.1993	127.5	0.0177	0	0
50000	6.732	1022.04	0.1928	0	0
100000	12.7739	2291.61	0.3728	0	0

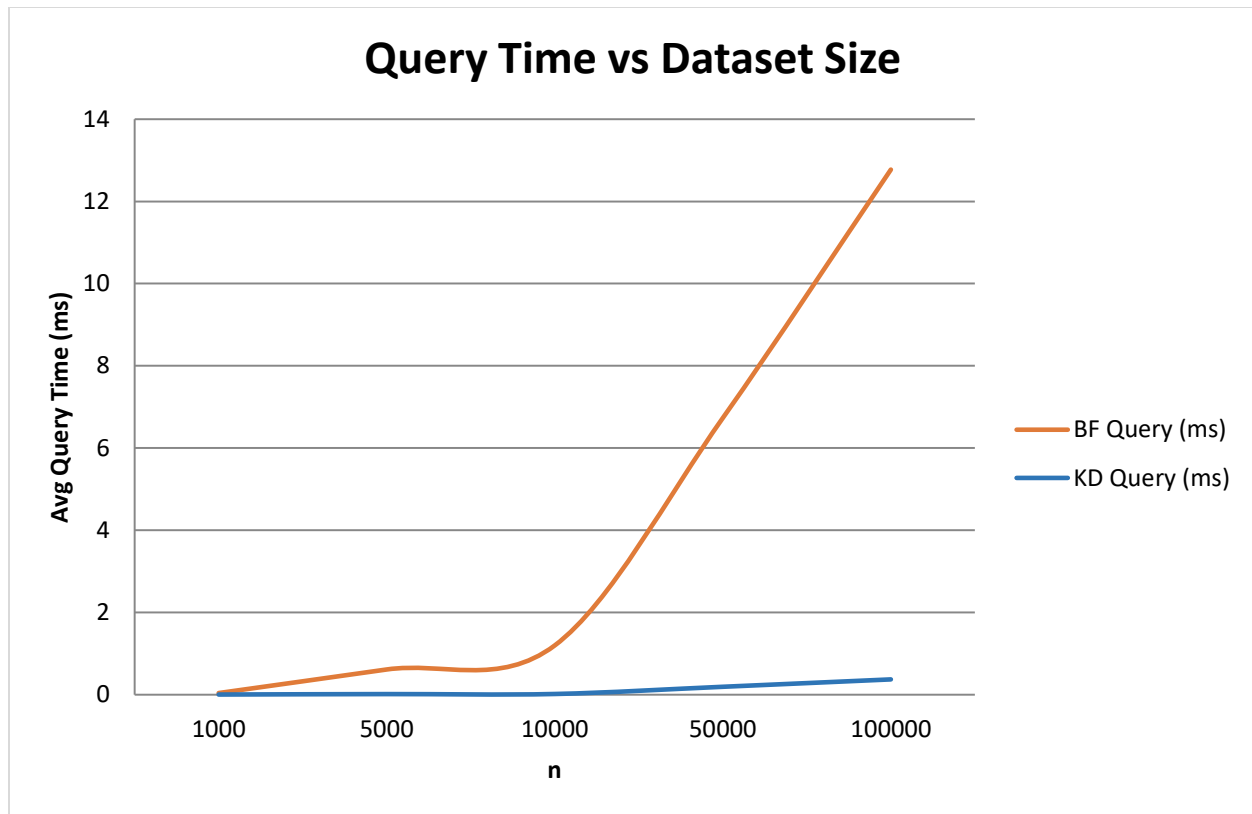


Figure 1: Query time vs. dataset size. Brute-force grows linearly with n ; KD-tree query stays nearly flat.

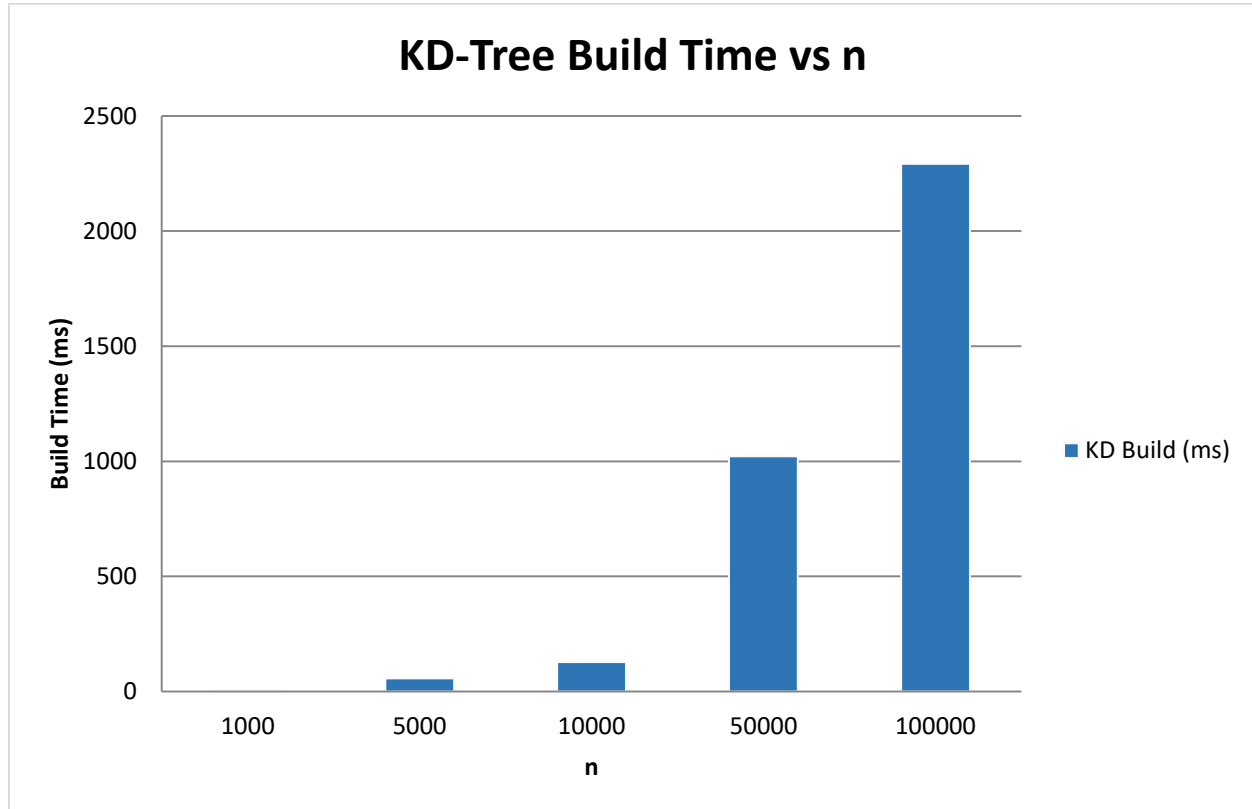


Figure 2: KD-tree build time vs. dataset size. Build cost grows significantly — over 2 seconds at $n=100000$.

From Figures 1 and 2, brute-force time grows from 0.04ms at $n=1000$ to 12.77ms at $n=100000$ (about $316\times$ more), confirming $O(n)$ growth. KD-tree query stays very fast (0.37ms at $n=100K$), but the build time is expensive, over 2.3 seconds for 100000 vectors. This shows KD-tree is only practical when many queries run on the same dataset.

6.2 Experiment 2 — Varying Dimensions ($n=10K$, $r=1.5$)

Experiment 2: Varying Dimensions ($n=10000$, $r=1.5$)					
Param	BF Query (ms)	KD Build (ms)	KD Query (ms)	BF Found	KD Found
4	0.4879	102.12	0.0303	22.08	22.08
8	0.7861	111.76	0.0177	0	0
16	1.1455	121.09	0.0202	0	0
32	2.4437	143.19	0.1779	0	0
64	4.6392	241.95	0.1675	0	0
128	10.5442	323.27	0.1949	0	0

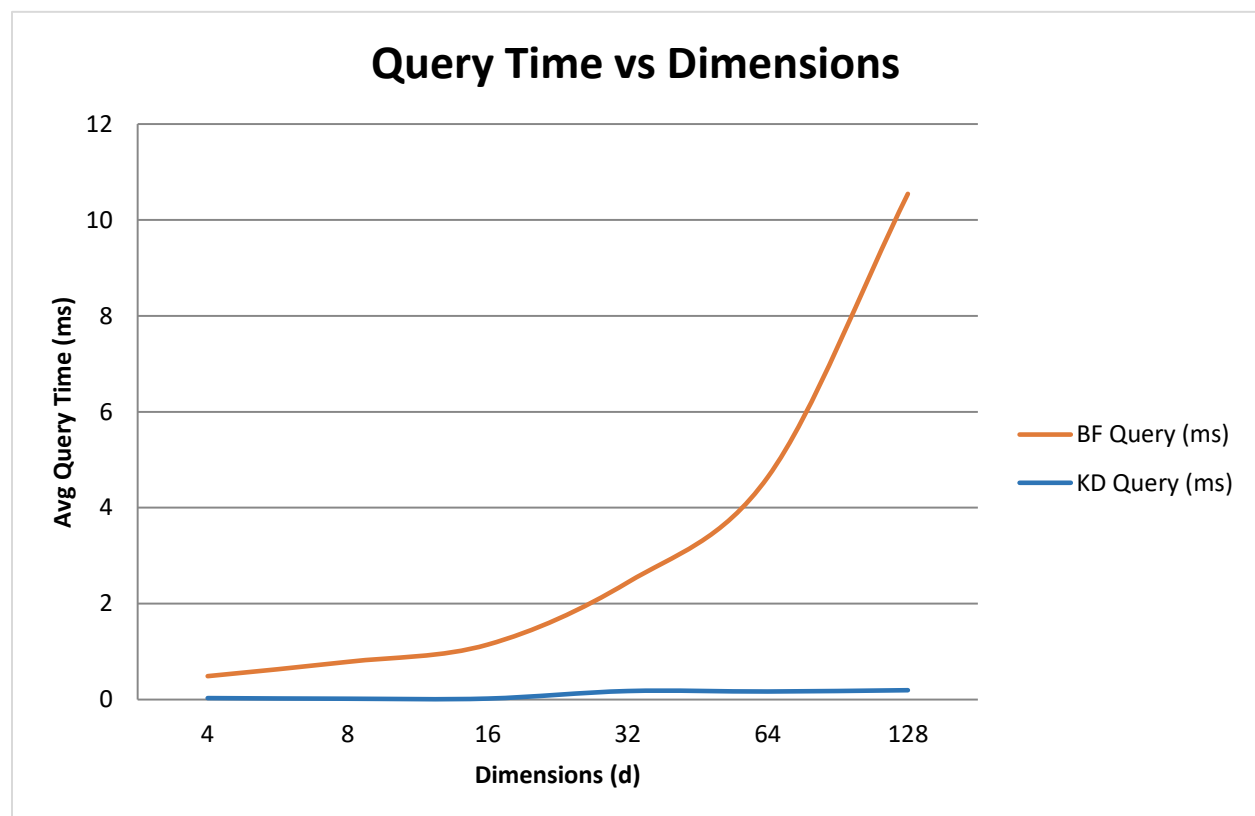


Figure 3: Query time vs. dimensions. Brute-force grows much faster than KD-tree. Both slow down as d increases.

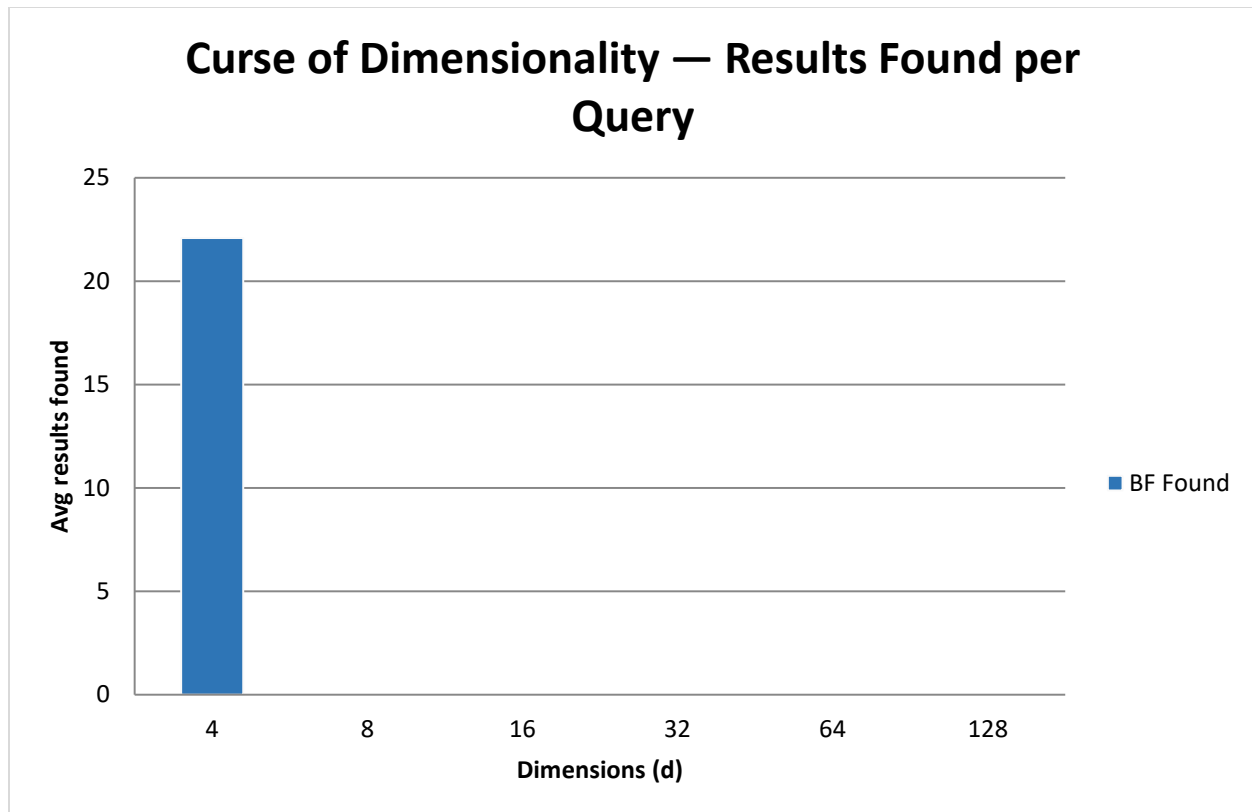


Figure 4: Curse of dimensionality. At $d=4$, 22 results found per query. From $d=8$ onwards: 0 results with fixed $r=1.5$.

This experiment had the most interesting result. At $d=4$, both methods found about 22 results per query. But from $d=8$, the results dropped to 0, this is the curse of dimensionality. As dimensions increase, the fixed radius $r=1.5$ covers almost no volume of the search space, so no neighbours are found. Brute-force time also grows significantly with d ($0.49\text{ms} \rightarrow 10.54\text{ms}$), confirming $O(d)$ per distance calculation.

6.3 Experiment 3 — Varying Search Radius ($n=10K$, $d=16$)

Experiment 3: Varying Search Radius ($n=10000$, $d=16$)					
r (radius)	BF Query (ms)	KD Build (ms)	KD Query (ms)	BF Found	KD Found
0.5 (small)	1.4286	136.48	0.1455	0	0
1.5 (medium)	1.1146	119.94	0.1682	0	0
3.0 (large)	1.2576	117.78	0.5454	0	0

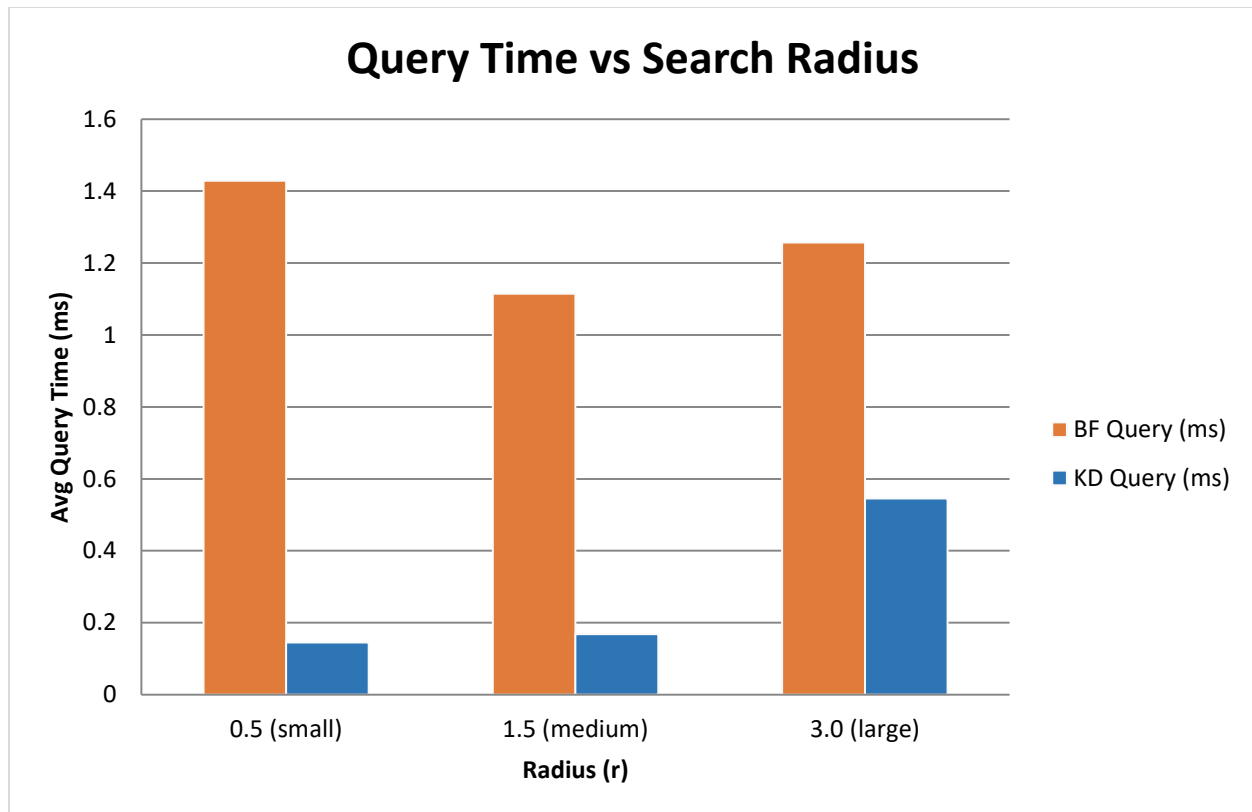


Figure 5: Query time vs. search radius. KD-tree time grows with larger r because pruning activates less often.

Changing the radius did not affect brute-force much since it always checks all vectors. For KD-tree, a larger radius means the pruning condition $|\text{diff}| \leq r$ is satisfied more often, so more branches must be explored. That is why KD-tree query time increased from 0.15ms to 0.55ms as r went from 0.5 to 3.0.

7. Discussion

7.1 Which Method is Better?

For low-dimensional data ($d \leq 16$), KD-tree queries are clearly faster per query. At $d=4$, brute-force takes 0.49ms while KD-tree takes 0.03ms, about $16\times$ faster. However, the build cost (102ms) is significant. KD-tree makes sense only when many queries run on the same dataset.

For high dimensions ($d \geq 32$), the gap shrinks. At $d=128$, brute-force is 10.54ms and KD-tree is 0.19ms per query, but the build is 323ms. In production AI systems with 768+ dimensional embeddings, approximate methods like HNSW or LSH would be more practical.

7.2 Curse of Dimensionality

Figure 4 shows the effect clearly. With $r=1.5$ and uniform data in $[0,10]$, only $d=4$ returns results. At $d \geq 8$, zero results. The geometric reason: the volume of a d -dimensional hypersphere is proportional to $r^d / \Gamma(d/2+1)$, which shrinks relative to the total space as d grows. In practice, all points become roughly equidistant in high dimensions, making fixed-radius search ineffective.

- KD-tree pruning also degrades: $|\text{diff}| \leq r$ is almost always true in high-d, so no branches get skipped.
- To find neighbours in high-d spaces, you need to either use a much larger r or switch to approximate methods.

7.3 Theory vs. Measurements

- Brute-force $O(n \times d)$: doubling n doubles time, doubling d doubles time
- KD-tree $O(\log n)$ for low-d: query time grows slowly with n at $d=4$
- KD-tree degrades in high-d: query time increases from $d=32$ to $d=128$
- KD-tree build $O(n \log n)$: build time grows faster than linearly with n

8. Conclusion

In this lab I implemented brute-force and KD-tree range search and compared them across different dataset sizes, dimensions, and radius. The KD-tree is clearly better for low-dimensional data, but its advantage disappears as dimensions grow due to the curse of dimensionality.

The most striking finding was that with $n=10,000$ vectors and $r=1.5$, we found 22 neighbors at $d=4$ but zero at $d=8$ and above. This shows how different high-dimensional geometry is from what we intuitively expect.

For real AI applications with high-dimensional vectors like sentence embeddings, exact KD-tree search is not practical. Approximate methods would be needed to make range search efficient at scale.

Appendix: How to Run

Compile:

MSYS2 UCRT64 Terminal

```
g++ -std=c++20 lab3_search.cpp -o lab3_search
```

Run:

```
./lab3_search
```

Output: console tables + lab3_results.csv